langsmith fleet × snowflake

Querying Snowflake from LangSmith Fleet

How a Fleet agent reaches governed Snowflake data through a managed MCP server, and how each person's identity and roles carry across both systems.

Live demo on TPC-H sample data · → or space to advance

start with the answer

you · in LangSmith Fleet

What was total revenue by region in 1995?

Snowflake agent

Europe led 1995 revenue at $6.70B; all five regions were within about 2% of each other.

Europe	$6,701,964,431
Asia	$6,664,790,148
Africa	$6,642,990,084
America	$6,639,656,068
Middle East	$6,567,494,315

How did that happen?

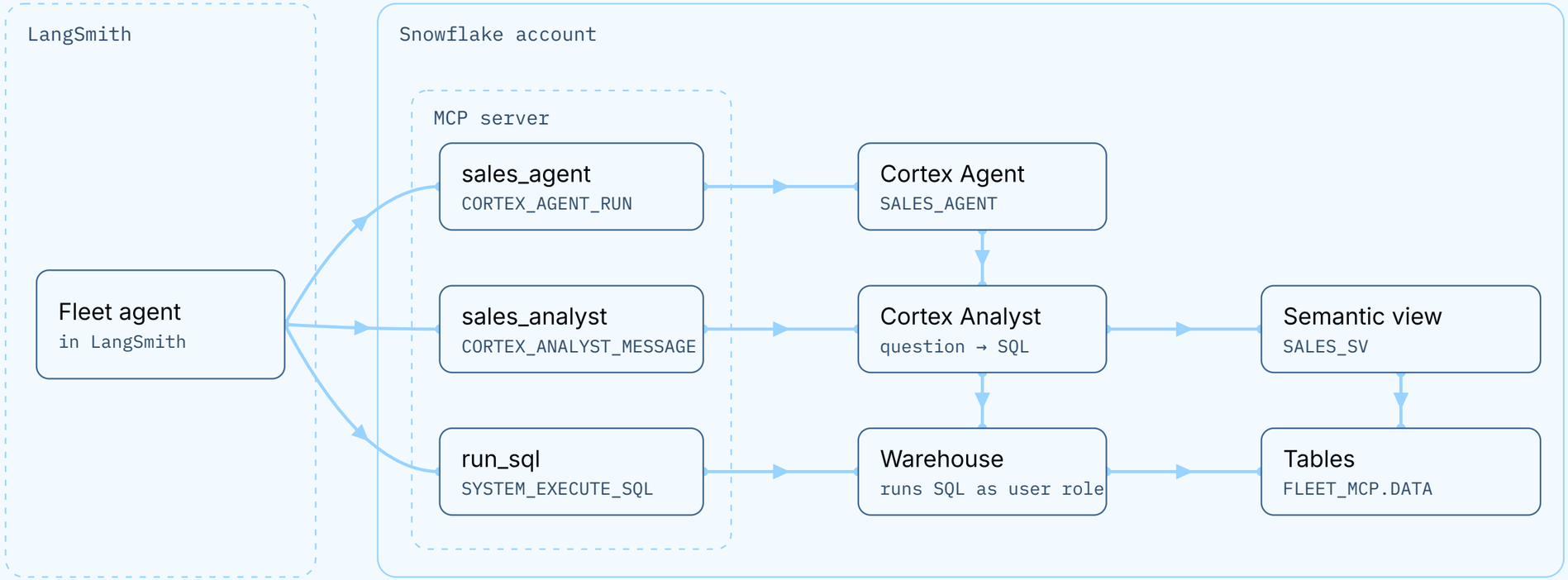
> Where does "revenue" get its definition?
> the semantic layer

> What did Fleet actually call in Snowflake?
> the MCP server and its tools

> Whose permissions did the query use?
> OAuth and Snowflake roles

the system

One map for the whole talk



Fleet agent

Runs in LangSmith. Sees Snowflake as MCP tools on one connection.

MCP server

A Snowflake object that lists which tools Fleet may call. You create and grant it.

Cortex Agent and Analyst

Snowflake's AI services. Analyst writes SQL; the Agent plans, runs SQL and answers.

Semantic view

Business definitions over your tables. Everything above reads it.

the semantic layer

Teaching Snowflake what "revenue" means

What the tables say

LINEITEM.L_EXTENDEDPRICE

LINEITEM.L_DISCOUNT

ORDERS.O_ORDERDATE

LINEITEM.L_SHIPDATE

NATION.N_REGIONKEY

REGION.R_NAME

→

What SALES_SV defines

total_revenue	SUM(L_EXTENDEDPRICE * (1 - L_DISCOUNT))
	synonyms: revenue, sales, net sales
region_name	REGION.R_NAME via nation → region joins
order_year	YEAR(O_ORDERDATE)

What we hit in testing

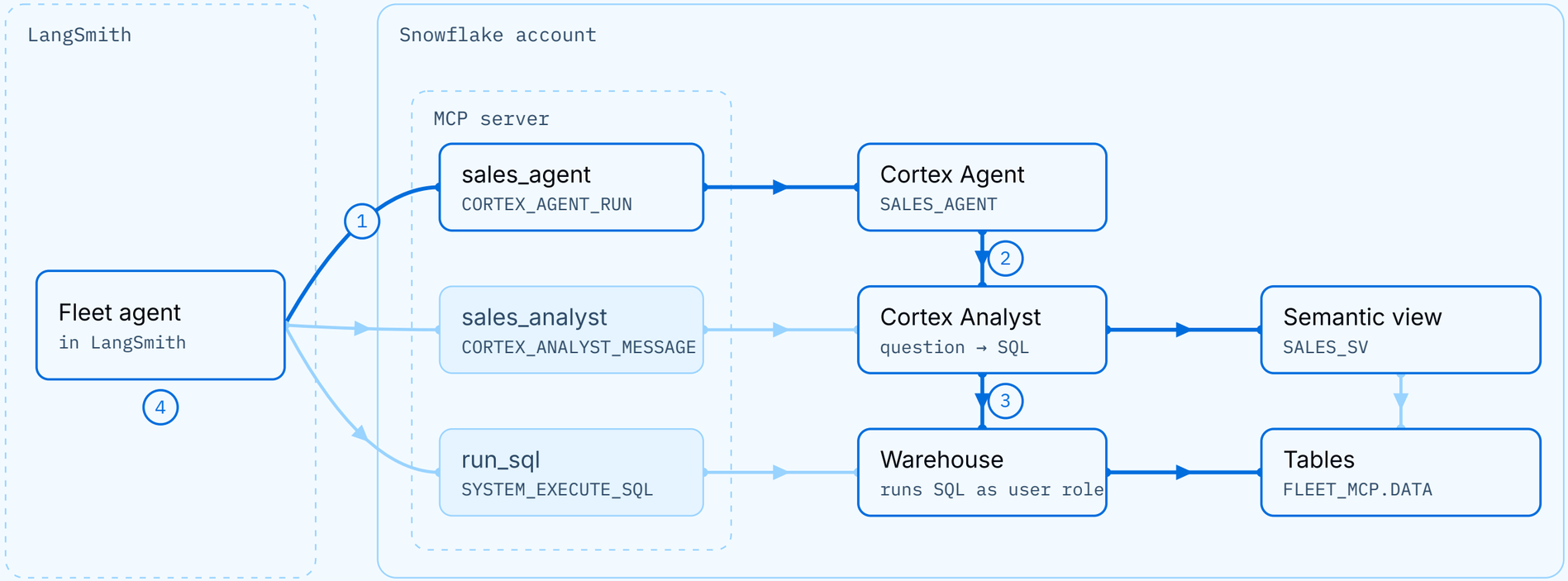
Asked for 1995 revenue, Analyst filtered on `L_SHIPDATE` while the Agent used `O_ORDERDATE`. Europe came back as `$6.718B` vs `$6.702B`.

One rule fixed both

"Year questions use order date" added to the semantic view. Every agent and tool that reads it now agrees. Define it once, inherit it everywhere.

pattern a

Fleet calls a Cortex Agent



1 **Fleet calls one tool**
`sales_agent` with the user's question.

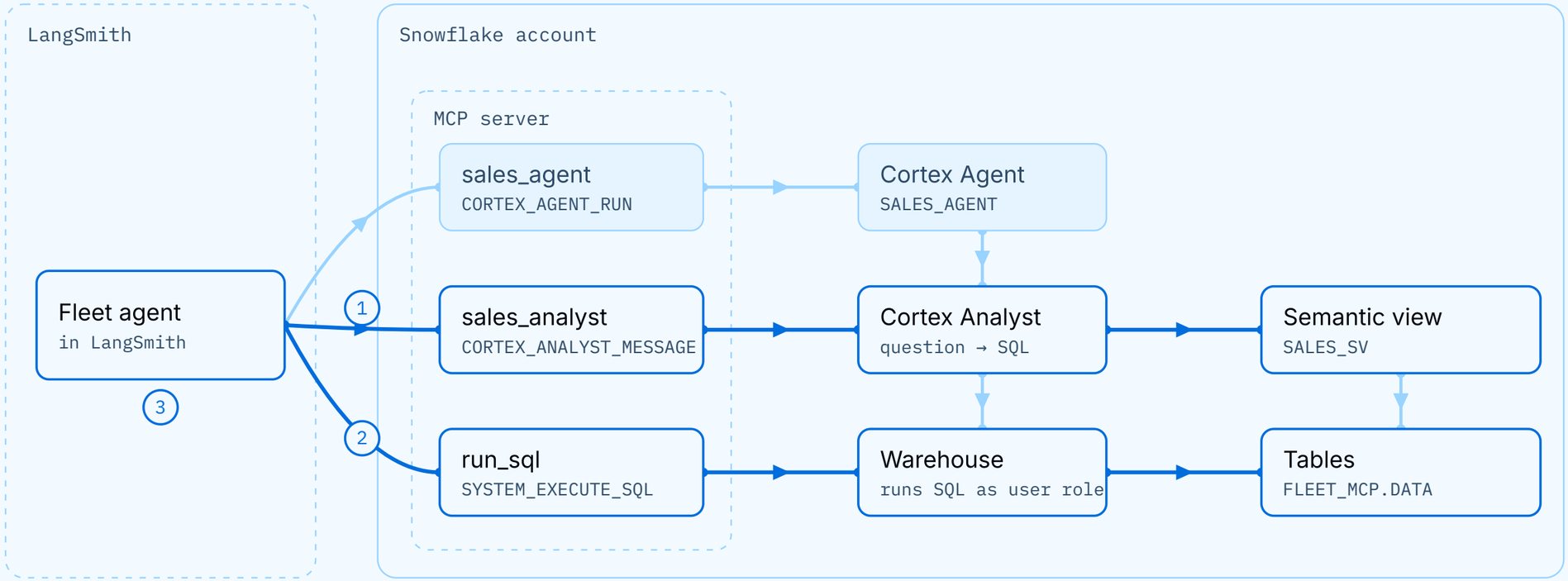
2 **The Agent plans**
It asks Analyst, which writes SQL from the semantic view.

3 **It runs the SQL**
On the warehouse, as the signed-in user's role.

4 **One big reply**
Answer, rows, chart spec and follow-ups. ~18 s, ~31 KB.

pattern b

Fleet calls Analyst, then runs the SQL



1 **Get the SQL**
`sales_analyst` reads the semantic view and returns SQL only. ~4 s.

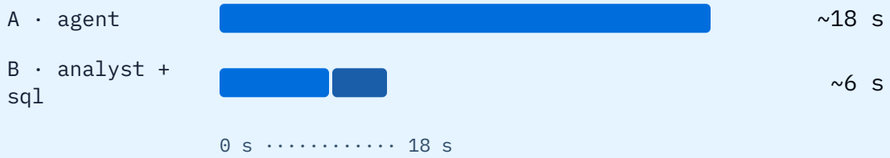
2 **Run it**
Fleet passes that SQL to `run_sql` (read-only). Rows come back. ~2 s.

3 **Fleet writes the answer**
Same numbers as pattern A. Every step and the SQL are in the LangSmith trace.

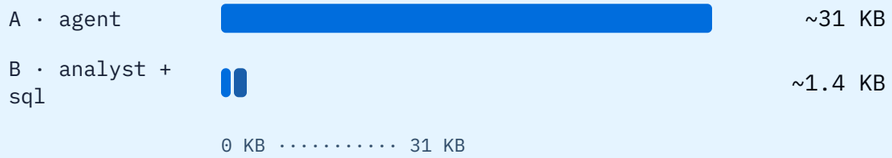
choosing a pattern

Same answer, different trade-offs

Time to answer



Data sent back to Fleet



	A · Fleet → Cortex Agent	B · Fleet → Analyst → SQL
who reasons	The Cortex Agent, inside Snowflake	The Fleet agent
in the trace	One tool call; internals stay in Snowflake	Every step, including the generated SQL
use when	A governed Cortex Agent is the product, or you want Snowflake to combine Analyst, Search and charts	Fleet mixes Snowflake with other tools and you want speed and full traceability

Measured on one question with an XS warehouse; indicative only.

live demo

Same question, both agents

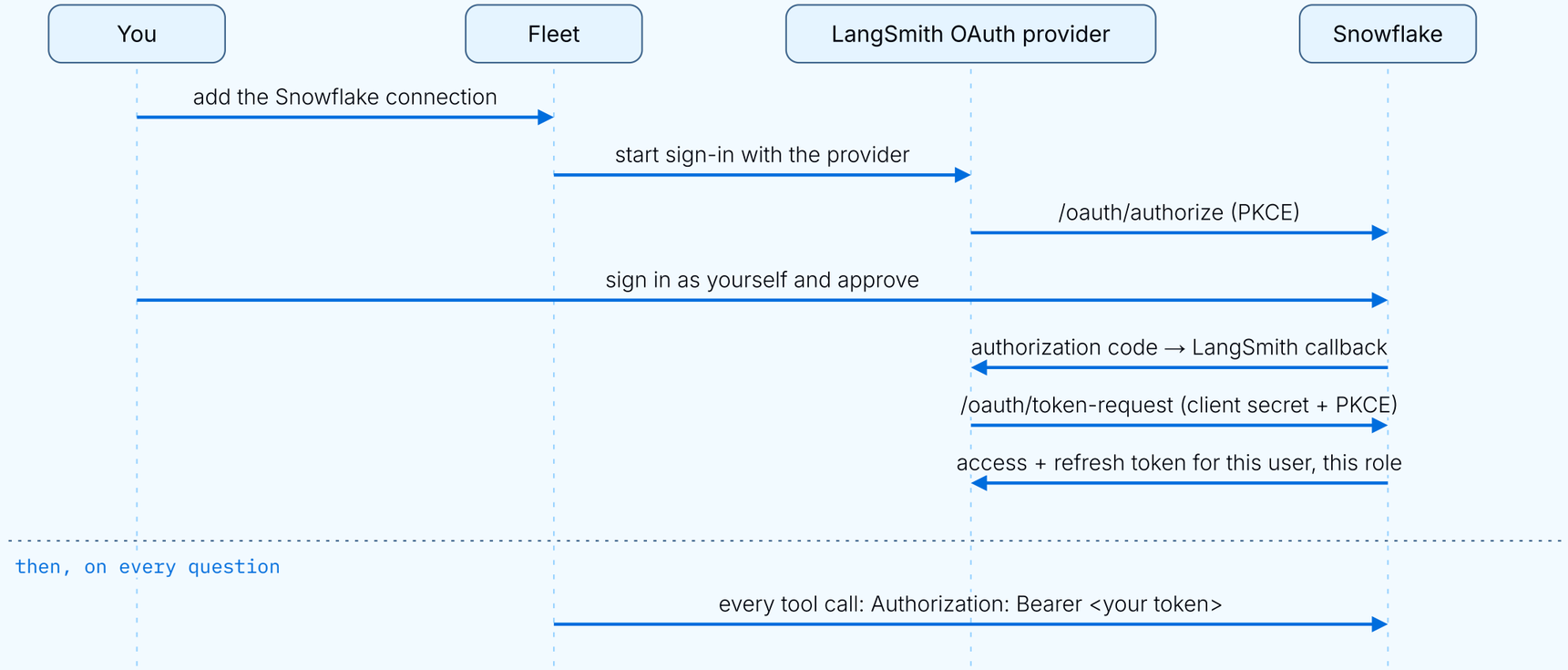
1. **"What was total revenue by region in 1995?"**
   Ask both Fleet agents. Expect Europe highest at about $6.70B and Middle East lowest at about $6.57B.

2. **Open both traces in LangSmith**
   Pattern A: one long tool call. Pattern B: two short calls with the SQL visible.

3. **"What's our employee headcount?"**
   Not in the sales data. Both agents should say so instead of guessing.

authentication

Each person signs in to Snowflake once



1 · connect

You add the Snowflake MCP server in Fleet. Fleet hands off to the LangSmith OAuth provider, which sends your browser to Snowflake's sign-in page. PKCE ties this request to the code that comes back, so it can't be intercepted and reused.

2 · sign in

You sign in on Snowflake's own page, with your Snowflake credentials or SSO, and approve access. LangSmith and Fleet never see your password.

3 · exchange

Snowflake sends a one-time code to LangSmith's callback URL. LangSmith trades it, plus the client secret, for an access token and a refresh token issued to you, under an allowed role.

4 · every question

Each Fleet tool call carries your token. Snowflake runs the session as your role, so your grants decide what every query can reach. The refresh token renews access without signing in again (up to a day in this setup).

access control

How Snowflake roles are respected

gate 1 · oauth integration

Which roles may sign in

`ALLOWED_ROLES_LIST` names the roles Fleet sessions may use. ACCOUNTADMIN and other admin roles are blocked by default.

→

gate 2 · session role

One role per session

The session runs as the user's role, from the token's scope or their default. `OAUTH_USE_SECONDARY_ROLES = NONE` stops other roles leaking in.

→

gate 3 · object grants

Each tool checks grants

`USAGE` on the agent, `SELECT` on the semantic view and tables. `run_sql` is read-only.

→

Data

Only what that role can read.

Same Fleet agent, same MCP server, two people · design example, not part of today's demo

signed-in user	role	sales_analyst	finance_analyst	run_sql on finance tables
Sales analyst	SALES_READ	works	no privilege	no privilege
Finance lead	FINANCE_READ	works	works	works

LangSmith decides which tools an agent can call. Snowflake decides what data each person can reach. Neither system has to copy the other's permissions.

beyond one demo

General-purpose chat across many databases

Give each business domain a semantic view over its own database or schemas. Each Analyst tool covers exactly one view (we tested this), so something has to pick the right view per question.

pattern b Fleet routes

fleet agent · picks tools by description

mcp server · up to 50 tools

`sales_analyst` `finance_analyst` `support_analyst` `run_sql`

semantic views → databases

`SALES_SV → SALES_DB` `FINANCE_SV → FIN_DB` `SUPPORT_SV → SUPPORT_DB`

Fleet's reasoning and every SQL statement show in the trace.

pattern a Snowflake routes

fleet agent · sees one tool

mcp server → cortex agent · combines results

`Analyst → SALES_SV` `Analyst → FINANCE_SV` `Analyst → SUPPORT_SV` `Search → docs`

semantic views → databases

`same views, same grants`

Tested: asked for customers and suppliers per region, the agent queried two views and combined them (~35 s).

Split servers by access

A server is an access boundary: e.g. `ANALYTICS_MCP` for everyone, `FINANCE_MCP` for finance roles.

No "view of views"

The router combines domains. Add a broader view only where cross-domain questions are common.

Shared dimensions

Define region, customer and date the same way in every view so answers line up.

takeaways

Setting this up yourself

1
Semantic views
One per domain, with metrics, synonyms and SQL rules.

2
Cortex Agent
Optional for pattern B. One Analyst tool per view.

3
MCP server
Choose the tools. Split servers by who may use them.

4
Roles and grants
Server, agent, views and tables. Default role and warehouse per user.

5
OAuth
Snowflake integration ⇄ LangSmith provider ⇄ Fleet connection.

Start with pattern B when Fleet owns the reasoning, mixes in other tools, and you want every step traced.

Use pattern A where a governed Cortex Agent is the product, or Snowflake should orchestrate Analyst, Search and charts.

In production, put run_sql on its own server and role. Never grant MCP roles to PUBLIC.